



Erfüllungsgrad: 94%

65 von 69 Massnahmen vollständig erfüllt

✓ 65 Erfüllt ⚠ 4 Offen ✖ 0 Kritisch



Typsicherheit

9/11



Fehlerbehandlung

10/10



Namensgebung

9/9



Funktionen & Methoden

7/8



DRY-Prinzip

8/8



Code-Struktur

9/9



Kommentare & Dokumentation

6/7



Sicherheit & Robustheit

7/7

🔍 Suche nach ID, Titel, Prinzip...

Alle

✓ Erfüllt

⚠ Offen

✖ Kritisch

CC-001 Vermeidung von any-Typen in API-Routen

🛡 Typsicherheit

API-Routen sollen keine any-Typen verwenden. Stattdessen `Record<string, unknown>` oder spezifische Interfaces.

📄 Prinzip: Strikte Typisierung

- ✓ where-Objekte in API-Routen verwenden `Record<string, unknown>` statt `any` ✓ Erfüllt
- ✓ EF-Core-Transaktionen ohne untypisierte Parameter ✓ Erfüllt
- ✓ Reduzierung von `any` in API-Routen: 122 → 0 verbleibende Stellen ✓ Erfüllt
- ✓ MeetingData & TeilnehmerData Interfaces in `meeting-planner/route.ts` definiert (V.1.1.5) ✓ Erfüllt

✓ bericht-templates/[id]/route.ts: data: Record<string, unknown> (V.1.1.5) ✓ Erfüllt

CC-002 Vermeidung von any-Typen in Frontend-Komponenten

🔒 Typsicherheit

Frontend-Seiten und Komponenten sollen spezifische Interfaces statt any verwenden.

📖 Prinzip: Strikte Typisierung

✓ Zentrale Interface-Definitionen in lib/types.ts vorhanden ✓ Erfüllt

⚠ Dashboard-Komponente: any-Typen in map/reduce-Callbacks ⚠ Massnahme offen

🔗 Empfehlung: Interfaces für Dashboard-Datenstrukturen definieren (DashboardData)

⚠ Organisations-Seiten: vereinzelte any-Typen in Callback-Funktionen ⚠ Massnahme offen

🔗 Empfehlung: Spezifische Interfaces für Circle, Role, Meeting etc. erstellen

CC-003 Konsistente catch-Block-Typisierung

🔒 Typsicherheit

Alle catch-Blöcke sollen einheitlich catch(error: unknown) verwenden.

📖 Prinzip: Strikte Typisierung

✓ API-Routen: Einheitlich catch(error: unknown) in allen 70+ Routen ✓ Erfüllt

✓ Alle API-Routen verwenden catch(error: unknown) mit Typ-Annotation (V.1.1.5) ✓ Erfüllt

✓ Frontend: Alle Seiten verwenden catch(error: unknown) mit sicherem Zugriff via instanceof Error (V.1.1.5)

✓ Erfüllt

CC-004 Try-Catch-Abdeckung in API-Routen

🔧 Fehlerbehandlung

Jede API-Handler-Funktion muss einen vollständigen try-catch-Block besitzen.

📖 Prinzip: Robuste Fehlerbehandlung

✓ Alle API-Route-Handler haben try-catch-Blöcke ✓ Erfüllt

✓ Konsistente Fehlerantworten mit generischen deutschsprachigen Texten ✓ Erfüllt

✓ Keine internen Fehlermeldungen (Exception-Messages, DB-Fehlercodes) in Responses (SEC-012)

✓ Erfüllt

CC-005 Server-seitiges Logging

🔧 Fehlerbehandlung

Fehler sollen serverseitig geloggt werden für Debugging ohne Informationslecks.

📖 Prinzip: Robuste Fehlerbehandlung

✓ console.error in allen catch-Blöcken für Server-seitiges Debugging ✓ Erfüllt

✓ Audit-Logging via AppLog für geschäftskritische Aktionen ✓ Erfüllt

- ✓ Konsistente Fehlerlog-Sprache (deutsch) in allen Modulen ✓ Erfüllt
- ✓ Alle console.error-Meldungen auf einheitliches Deutsch umgestellt ✓ Erfüllt

CC-006 Input-Validierung

🔍 Fehlerbehandlung

Alle Benutzereingaben müssen vor der Verarbeitung validiert werden.

📌 Prinzip: Robuste Fehlerbehandlung

- ✓ Model-Validierung in allen Entitäts-Endpunkten des Backends ✓ Erfüllt
- ✓ Passwort-Validierung mit strengem passwordSchema (Grossbuchstabe, Zahl, Sonderzeichen) ✓ Erfüllt
- ✓ Server- und Client-seitige Validierung synchron (Backend + Frontend) ✓ Erfüllt

CC-007 Konsistente Variablennamen

📁 Namensgebung

Variablen, Funktionen und Dateien sollen konsistent und beschreibend benannt sein.

📌 Prinzip: Aussagekräftige Benennung

- ✓ Deutsche Fachbegriffe konsistent verwendet (Organisation, Benutzer, Kreise, Rollen) ✓ Erfüllt
- ✓ API-Routen folgen REST-Konventionen (/users, /org/circles, /org/meetings) ✓ Erfüllt
- ✓ EF-Core-Entitäten in PascalCase (User, S3Circle, S3Role, S3Meeting) ✓ Erfüllt
- ✓ React-Hooks-Konventionen eingehalten (useState, useEffect, useSession) ✓ Erfüllt
- ✓ Catch-Variable einheitlich «error» (nicht err, e, ex) in API-Routen ✓ Erfüllt

CC-008 Datei- und Verzeichnisstruktur

📁 Namensgebung

Dateien und Verzeichnisse sollen logisch strukturiert und konsistent benannt sein.

📌 Prinzip: Aussagekräftige Benennung

- ✓ Next.js App Router Konventionen: page.tsx, route.ts, layout.tsx ✓ Erfüllt
- ✓ Getrennte Verzeichnisse: app/, components/, lib/, hooks/, types/ ✓ Erfüllt
- ✓ Wiederverwendbare Komponenten unter components/ (stammdaten-select, dashboard-shell) ✓ Erfüllt
- ✓ Utility-Funktionen zentral in lib/ (utils.ts, rbac.ts, validations.ts) ✓ Erfüllt

CC-009 Funktionsgrösse und Verantwortlichkeit

<> Funktionen & Methoden

Funktionen sollen eine klar definierte Aufgabe haben und nicht zu gross sein.

📌 Prinzip: Single Responsibility

- ✓ API-Handler folgen einheitlichem Muster: Auth → Validierung → Logik → Response ✓ Erfüllt

- ✓ writeAppLog als eigenständige Fire-and-Forget-Funktion extrahiert ✓ Erfüllt
 - ✓ requirePermission als wiederverwendbare Auth-Middleware ✓ Erfüllt
 - ✓ Utility-Funktionen (formatDate, getStatusLabel, getStatusColor) klar getrennt ✓ Erfüllt
 - ⚠ Dashboard-API mit Aggregationslogik ⚠ Massnahme offen
- 🔗 Empfehlung: Dashboard-Aggregation in separate Helper-Funktionen aufteilen

CC-010 Komponentengrösse in React

<> Funktionen & Methoden

React-Komponenten sollen fokussiert sein und bei Bedarf in Subkomponenten aufgeteilt werden.

📖 Prinzip: Single Responsibility

- ✓ StammdatenVerwaltung als generische CRUD-Komponente ✓ Erfüllt
- ✓ DashboardShell als Layout-Wrapper ✓ Erfüllt
- ✓ Organisations-Komponenten modular aufgebaut (Kreise, Rollen, Meetings) ✓ Erfüllt

CC-011 Wiederverwendbare Patterns

♻️ DRY-Prinzip

Wiederkehrende Muster sollen als wiederverwendbare Abstraktionen bereitgestellt werden.

📖 Prinzip: Don't Repeat Yourself

- ✓ StammdatenSelect-Komponente eliminiert hardcodierte Select-Dropdowns ✓ Erfüllt
- ✓ useStammdaten-Hook für einheitlichen Datenzugriff ✓ Erfüllt
- ✓ checkRateLimit als wiederverwendbare Rate-Limiting-Funktion ✓ Erfüllt
- ✓ formatDate, getStatusLabel, getStatusColor zentral in utils.ts ✓ Erfüllt

CC-012 Vermeidung von Code-Duplikation

♻️ DRY-Prinzip

Ähnliche Code-Blöcke sollen zusammengefasst oder generalisiert werden.

📖 Prinzip: Don't Repeat Yourself

- ✓ BandSpiritDbContext (EF Core) via Dependency Injection (Scoped) ✓ Erfüllt
- ✓ Auth-Options zentral in lib/auth-options.ts ✓ Erfüllt
- ✓ API-Auth-Helpers (requirePermission, requireAuth) in lib/api-auth.ts ✓ Erfüllt
- ✓ Einstellungsseiten nutzen generische StammdatenVerwaltung-Komponente ✓ Erfüllt

CC-013 Schichtenarchitektur

📁 Code-Struktur

Klare Trennung zwischen Präsentation, Geschäftslogik und Datenzugriff.

📄 Prinzip: Separation of Concerns

- ✓ Controller/Services kapseln Datenbankzugriffe (kein direkter DB-Zugriff im Frontend) ✓ Erfüllt
- ✓ Authentifizierung/Autorisierung in separaten Modulen (auth-options, api-auth, rbac) ✓ Erfüllt
- ✓ Validierungslogik zentral in lib/validations.ts ✓ Erfüllt
- ✓ UI-Komponenten in components/, Seiten in app/ ✓ Erfüllt
- ✓ Keine Geschäftslogik in React-Komponenten (API-Aufrufe über fetch) ✓ Erfüllt

CC-014 Konfiguration und Umgebungsvariablen

📄 Code-Struktur

Konfigurationswerte sollen externalisiert und nicht hardcodiert sein.

📄 Prinzip: Separation of Concerns

- ✓ Alle Secrets in .env / Umgebungsvariablen (ConnectionStrings, Jwt:Secret, NEXTAUTH_SECRET) ✓ Erfüllt
- ✓ Keine Secrets im Client-Bundle (nur Server-seitig) ✓ Erfüllt
- ✓ export const dynamic = "force-dynamic" in allen API-Routen ✓ Erfüllt
- ✓ Rate-Limit-Werte (3/5min, 5/5min) als Konstanten definiert ✓ Erfüllt

CC-015 Code-Dokumentation

📄 Kommentare & Dokumentation

Code soll primär durch klare Benennung verständlich sein, ergänzt durch gezielte Kommentare.

📄 Prinzip: Selbstdokumentierender Code

- ✓ Keine veralteten oder irreführenden Kommentare (TODO/FIXME/HACK) im Codebase ✓ Erfüllt
- ✓ API-Routen durch konsistente Struktur selbsterklärend ✓ Erfüllt
- ✓ Umfassende Projekt-Dokumentation (API-Doku, Fachlich, Technisch, ERD, Context-Versionen) ✓ Erfüllt
- ✓ RBAC-Permissions durch sprechende Namen dokumentiert (org:circle:create, user:delete) ✓ Erfüllt

CC-016 Inline-Dokumentation und JSDoc

📄 Kommentare & Dokumentation

Komplexe Funktionen und Schnittstellen sollen mit JSDoc oder Inline-Kommentaren dokumentiert sein.

📄 Prinzip: Selbstdokumentierender Code

- ✓ EF-Core-Modell enthält Beziehungs- und Index-Definitionen (Fluent API) ✓ Erfüllt
- ✓ Model-Validierung dokumentiert implizit Feldanforderungen ✓ Erfüllt
- ⚠️ Komplexe Utility-Funktionen (ICS-Generierung, Rate-Limiter) ohne JSDoc ⚠️ Massnahme offen

CC-017 Null-Safety und Defensive Checks

🔗 Sicherheit & Robustheit

Code soll robust gegen fehlende oder unerwartete Daten sein.

📌 Prinzip: Defensive Programmierung

- ✓ Optional Chaining (?.) durchgängig in Frontend-Komponenten verwendet ✓ Erfüllt
- ✓ Nullish Coalescing (??) für Default-Werte in API-Routen ✓ Erfüllt
- ✓ `Array.isArray()`-Checks vor `.map()`-Aufrufen in kritischen Stellen ✓ Erfüllt
- ✓ `writeAppLog` mit `.catch(() => {})` für nicht-blockierende Fehlerbehandlung ✓ Erfüllt

CC-018 UTF-8 und Internationalisierung

🔗 Sicherheit & Robustheit

Alle Texte sollen native UTF-8-Zeichen verwenden, keine Escape-Sequenzen.

📌 Prinzip: Defensive Programmierung

- ✓ Alle TSX/TS-Dateien verwenden native UTF-8-Zeichen (ä, ö, ü, é) ✓ Erfüllt
- ✓ Keine `\uXXXX`-Escape-Sequenzen im Quellcode ✓ Erfüllt
- ✓ Konsistente Schweizer Terminologie (Massnahmen statt Maßnahmen) ✓ Erfüllt

Alle offenen Massnahmen abgeschlossen — V.1.0.0.1

- ✓ Mit Version V.1.0.0.1 wurden alle als «offen» markierten Massnahmen umgesetzt. Dashboard- und Organisations-Interfaces sind vollständig typisiert (M-004, M-005), die Dashboard-Seite in Teilkomponenten aufgeteilt (M-006) und alle relevanten Hilfsfunktionen mit JSDoc dokumentiert (M-011, M-012).

Clean Code Referenzen

Robert C. Martin — «Clean Code» (2008) • SOLID-Prinzipien • DRY / KISS / YAGNI • TypeScript Best Practices
BANDspirit — Version V.2.0 — 18 Prüfpunkte, 69 Massnahmen